Grant Permissions

The ROOK SDK requires that the user explicitly grant permissions to access and extract the data contained in Health Connect (Android) and Apple Health (iOS).

Request Permissions

`RookPermissions` is an object that provides functions useful for achieving this. The `requestAllPermissions` function requests permissions to access data, displaying a screen to the user for the necessary permissions. The `checkAvailability` function ensures that the services (Health Connect and Apple Health, respectively) are running on the device.

Android Permissions events

`RookPermissions` emits their own events that you can listen by attaching a listener like this:

```
const listener = RookPermissions.addListener('io.tryrook.permissions.healthConnect',
(permissionStatus: BoolResult) => {
      console.log('event health connect permissions', permissionStatus.result);
    })
```

- `io.tryrook.permissions.android` Event name describing if all Android permissions were granted.
- `io.tryrook.permissions.healthConnect`: Event name describing if all Health Connect permissions were granted.

Samsung Health availability & permissions

Availability

Before proceeding further, ensure that the Samsung Health app is installed and ready to be used, call

```
const checkSamsungHealthAvailability: () => Promise<StringResult>;`:
```

Status	Description	What to do
INSTALLED	Samsung Health is installed and ready to be used.	Proceed to check permissions
NOT_INSTALLED	Samsung Health is not installed.	Prompt the user to install Samsung Health.
OUTDATED	The version of Samsung Health is too old.	Prompt the user to update Samsung Health.
DISABLED	Samsung Health is disabled.	Prompt the user to enable Samsung Health.
NOT_READY	The user didn't perform an initial process, such as agreeing to the Terms and Conditions.	Prompt the user to open and complete the onboarding process of Samsung Health.

Permissions

These are permissions used to extract data, each Samsung Health data type is subject to a different permission, below you can see the list of permissions ROOK needs:

- ACTIVITY_SUMMARY
- BLOOD_GLUCOSE
- BLOOD_OXYGEN
- BLOOD_PRESSURE
- BODY_COMPOSITION
- EXERCISE
- EXERCISE_LOCATION
- FLOORS_CLIMBED
- HEART_RATE
- NUTRITION

- SLEEP
- STEPS
- WATER_INTAKE

To request permissions call:

```
const requestSamsungHealthPermissions: (props: SamsungPermissionTypePromps) =>
Promise<StringResult>;
```

```
export type SamsungPermissionTypePromps = {
  types: Array<SamsungPermissionType>;
}

export type SamsungPermissionType =
| 'ACTIVITY_SUMMARY'
|'BLOOD_GLUCOSE'
|'BLOOD_OXYGEN'
|'BLOOD_PRESSURE'
|'BODY_COMPOSITION'
|'EXERCISE'
|'EXERCISE_LOCATION'
|'FLOORS_CLIMBED'
|'HEART_RATE'
|'NUTRITION'
|'SLEEP'
|'STEPS'
|'WATER_INTAKE'
```

To check permissions call:

```
const checkSamsungHealthPermission: (props: SamsungPermissionTypePromps) =>
Promise<boolean>;
```

The previous function will check for all permissions, to check if at least one permission is granted, call:

```
const checkSamsungHealthPermissionPartially: (props: SamsungPermissionTypePromps) =>
Promise<boolean>;
```

functions accept a set of `SamsungPermissionType` so you can only request/check the permissions your app truly needs.

We recommend adding a screen for this purpose, like this:

```
import {
  IonButtons,
  IonContent,
  IonHeader,
  IonPage,
  IonTitle,
  IonToolbar,
  IonButton,
  IonBackButton,
  IonList,
  IonItem,
} from "@ionic/react";
import "./Home.css";
import { RookPermissions } from "capacitor-rook-sdk";
import { useHistory } from "react-router-dom";
import { useEffect, useState } from 'react';

const Permissions: React.FC = () => {
  const history = useHistory();

  useEffect(() => {

    const listenerAndroid =
RookPermissions.addListener('io.tryrook.permissions.android', (permissionStatus:
BoolResult) => {
      console.log('Android connect permissions', permissionStatus.result);
    })

    const listener =
RookPermissions.addListener('io.tryrook.permissions.healthConnect',
(permissionStatus: BoolResult) => {
      console.log('event health connect permissions', permissionStatus.result);
    })

    return () => {
      console.log('remove parmission view')
    };
  }, []);

  const goBack = () => {
```

```
    history.goBack();
  };

  const handleRequestAllPermissions = async (): Promise<void> => {
    try {
      const result = await RookPermissions.requestAllPermissions();
      console.log("result", result);
    } catch (error) {
      console.log("error", error);
    }
  };

  const handleRequestAllAppleHealthPermissions = async (): Promise<void> => {
    try {
      const result = await RookPermissions.requestAppleHealthPermissions();
      console.log("result", result);
    } catch (error) {
      console.log("error", error);
    }
  };

  const handleRequestAllHealthConnectPermissions = async (): Promise<void> => {
    try {
      const status: RequestPermissionsStatusResult = await
RookPermissions.requestHealthConnectPermissions();
      console.log("result", status.result);
    } catch (error) {
      console.log("error", error);
    }
  };

  const hanleOpenIOSSettings = async (): Promise<void> => {
    try {
      const result = await RookPermissions.openIOSSettings();
      console.log("result", result);
    } catch (error) {
      console.log("error", error);
    }
  };

  const handleOpenHealthConnectSettings = async (): Promise<void> => {
    try {
      const result = await RookPermissions.openHealthConnectSettings();
      console.log("result", result);
    } catch (error) {
      console.log("error", error);
```

```
    }
  };

  const handleRequestAndroidBackgroundPermissions = async (): Promise<void> => {
    try {
      const status: RequestPermissionsStatusResult = await
RookPermissions.requestAndroidPermissions();
      console.log("result", status.result);
    } catch (error) {
      console.log("error", error);
    }
  };

  const handleRequestSamsungHealthPermission = async(): Promise<void> => {
    try {
      const permissions: Array<SamsungPermissionType> = [
        "ACTIVITY_SUMMARY",
        "BLOOD_GLUCOSE",
        "BLOOD_OXYGEN",
        "BLOOD_PRESSURE",
        "BODY_COMPOSITION",
        "EXERCISE",
        "EXERCISE_LOCATION",
        "FLOORS_CLIMBED",
        "HEART_RATE",
        "NUTRITION",
        "SLEEP",
        "STEPS",
        "WATER_INTAKE"];
      const result: StringResult = await
RookPermissions.requestSamsungHealthPermissions({
        types: permissions
      })
      setTitle('Permissions');
      setMessage(`permissons result ${result.result}`);
    } catch (error) {
      setTitle('Permissions Error');
      setMessage(`Permissions error ${error}`);
    }
    setIsOpen(true);
  }

  return (
    <IonPage>
      <IonHeader>
        <IonToolbar>
```

```
            <IonButtons slot="start">
              <IonBackButton defaultHref="/" />
            </IonButtons>
            <IonTitle>Permissions</IonTitle>
          </IonToolbar>
        </IonHeader>
        <IonContent fullscreen>
          <IonButton onClick={goBack}>Go Back</IonButton>
          <IonList>
            <IonItem>
              <IonButton onClick={handleRequestAllPermissions}>
                Request permissions
              </IonButton>
            </IonItem>

            <IonItem>
              <IonButton onClick={handleRequestAllAppleHealthPermissions}>
                Request apple health permissions iOS
              </IonButton>
            </IonItem>

            <IonItem>
              <IonButton onClick={handleRequestAllHealthConnectPermissions}>
                Request health connect permissions
              </IonButton>
            </IonItem>

            <IonItem>
              <IonButton onClick={handleRequestSamsungHealthPermission}>Request
Samsung Health Permissions</IonButton>
            </IonItem>

            <IonItem>
              <IonButton onClick={hanleOpenIOSSettings}>
                {" "}
                open iOS settings
              </IonButton>
            </IonItem>

            <IonItem>
              <IonButton onClick={handleOpenHealthConnectSettings}>
                open health connect settings
              </IonButton>
            </IonItem>

            <IonItem>
```

```
            <IonButton onClick={handleRequestAndroidBackgroundPermissions}>
              Request background permissions android
            </IonButton>
          </IonItem>
        </IonList>
      </IonContent>
    </IonPage>
  );
};

export default Permissions;
```

Request Android Background Permissions

We highly recommend requesting another set of permissions to sync data in the background for Android. **This step is only required for Android.** To achieve this, add the following function to the previous screen:

```
const handleRequestAndroidBackgroundPermissions = async (): Promise<void> => {
  try {
    const result = await RookPermissions.requestAndroidBackgroundPermissions();
    console.log("result", result);
  } catch (error) {
    console.log("error", error);
  }
};
```